



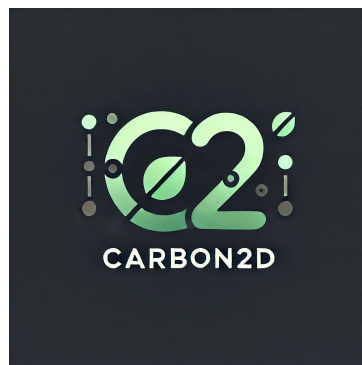
A.D. 1308
unipg
DIPARTIMENTO
DI INGEGNERIA

Tesina Finale di
Data Security and Blockchain
Corso di Laurea Magistrale in Ingegneria Informatica e Robotica – A.A. 2024-2025
DIPARTIMENTO DI INGEGNERIA

docente
Prof. Luca GRILLI

Carbon2D

Web DApp running on Sepolia Ethereum Testnet
Technology: Node.js/Next.js/TypeScript/Tailwind CSS/ethers.js



studenti

374446	Davide	Saia	davide.saia@studenti.unipg.it
368204	Daniele	Nanni Cirulli	daniele.nannicirulli@studenti.unipg.it

0. Indice

1	Descrizione del Problema	2
1.1	L'applicazione Carbon2D	3
1.2	Lo standard ERC-20	5
1.2.1	Funzioni Principali	5
1.2.2	Eventi	5
2	Specifica dei Requisiti	6
3	Progetto	8
3.1	Frontend della WebDapp	9
3.1.1	Struttura del Progetto	10
3.2	Home Page e Navigazione	11
3.3	Calcolatore delle Emissioni	12
3.4	Marketplace	13
3.5	Dashboard	14
3.6	Integrazione Blockchain e Smart Contract	15
3.6.1	Smart Contract	17
3.7	Problemi Riscontrati	21
4	Conclusioni e sviluppi futuri	23
5	Bibliografia	24

1. Descrizione del Problema

L'obiettivo di questo lavoro è lo sviluppo di una Web DApp, denominata *Carbon2D*, finalizzata alla creazione di token fungibili aderenti allo standard ERC-20 [7, 8] della blockchain Ethereum. La specificità di Carbon2D è quella di permettere di acquistare token per aderire a progetti per la riduzione di **emissioni** dell'anidride carbonica. Per iniziare, basta calcolare la propria **impronta** di carbonio e acquistare i token corrispondenti. I token sono acquistabili con la criptovaluta ETH.

La riduzione delle emissioni è un tema molto dibattuto a livello globale, ci sono molti esempi che cercano di ridurre la quantità di CO_2 grazie all'utilizzo delle tecnologie più innovative, come la **blockchain**. Un esempio importante sono organizzazioni che danno la possibilità di calcolare la propria impronta e aderire a progetti sia per la riduzione della CO_2 o progetti per la rimozione dei rifiuti e per la rigenerazione degli ambienti marini [4]. Per poter aderire a questi progetti si sfruttano spesso token virtuali che corrispondono a una certa quantità di CO_2 . In questo modo si vuole creare un legame diretto con l'ambiente, anche i progetti sono quantificati in token, ma i corrispettivi token con cui si può contribuire, rappresentano la quantità di CO_2 che si va a ridurre [3].

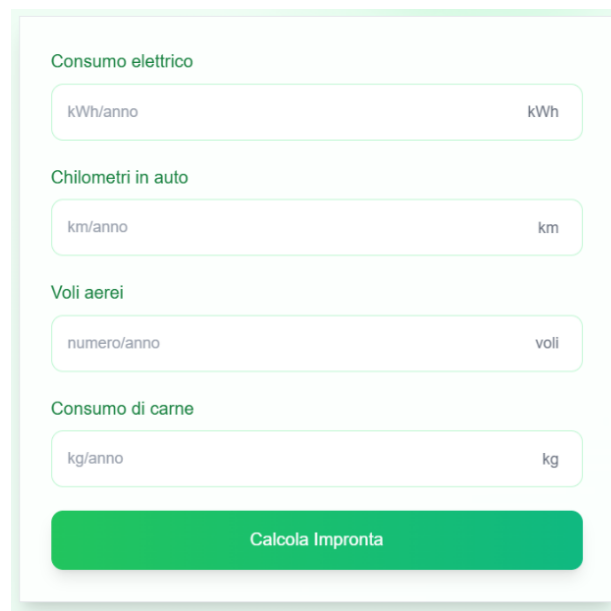
La Web DApp Carbon2D ha come caratteristica di implementare un server backend usando la tecnologia *Next.js*, che implementa in maniera ottimizzata sia la parte grafica, che la parte logica della pagina web. L'applicazione è interamente eseguibile in locale, installando specifiche dipendenze. Abbiamo deciso di implementare il progetto con l'uso di un interfaccia web, quindi eseguibile all'interno dei più comuni browser. Per quanto concerne la gestione dell'interfaccia utente grafica, l'applicazione sarà implementata utilizzando la tecnologia Node.js/Next.js/TypeScript/Tailwind CSS. Mentre sarà impiegata la libreria ethers.js [2] per gestire l'interazione con la blockchain. Lo smart contract della DApp sarà eseguito nella testnet pubblica Sepolia Ethereum [5], il linguaggio usato per l'implementazione del contratto è *Solidity*.

Di seguito sarà data una breve descrizione dell'applicazione Carbon2D e dello standard ERC-20, dopodiché si fornirà una descrizione generale della Web DApp Carbon2D che si intende realizzare.

1.1 L'applicazione Carbon2D

L'applicazione Carbon2D prende ispirazione dall'innovazione portata dalla **green economy**. È un tema molto delicato e che è al centro di molti dibattiti politici a livello mondiale. L'applicazione ha lo scopo di far partecipare l'utente a progetti mirati per la riduzione della CO_2 . L'app dà la possibilità di calcolare la propria impronta 1.1, attraverso una formula che prende come parametri quattro indicatori:

- Consumo elettrico (kWh)
- Km percorsi in auto
- Voli aerei
- Consumo di carne (kg)



The image shows a web form for calculating a carbon footprint. It is titled 'Form per il calcolo dell'impronta di carbonio'. The form contains four input fields, each with a label and a unit. The first field is 'Consumo elettrico' with unit 'kWh', the second is 'Chilometri in auto' with unit 'km', the third is 'Voli aerei' with unit 'voli', and the fourth is 'Consumo di carne' with unit 'kg'. All input fields are currently empty. Below the input fields is a green button labeled 'Calcola Impronta'.

Indicator	Unit
Consumo elettrico	kWh
Chilometri in auto	km
Voli aerei	voli
Consumo di carne	kg

Calcola Impronta

Figura 1.1: Form per il calcolo dell'impronta di carbonio

I parametri specificati si riferiscono a un anno solare. L'utente, a seguito del calcolo della propria impronta, può decidere di acquistare il numero di token corrispondenti ai Kg di CO_2 che ha generato in un anno. Ogni token corrisponde a $100kg$ di CO_2 .



Figura 1.2: Esempio di calcolo della propria impronta con risultanti token riscattabili

I progetti a cui l'utente può partecipare sono visualizzabili nella sezione **marketplace**. All'interno di questa sezione viene fornita una vista generale e intuitiva del progetto a cui si vuole aderire, attraverso una breve descrizione, informazioni riguardanti il numero di token necessari per il completamento del progetto e il continente in cui tale progetto verrà svolto. Inoltre la Dapp fornisce l'opportunità di poter creare un proprio progetto per la riduzione della CO_2 . Il progetto creato farà parte della lista dei progetti a cui si può contribuire. Al completamento dei progetti i token collezionati vengono inviati all'ideatore del progetto.

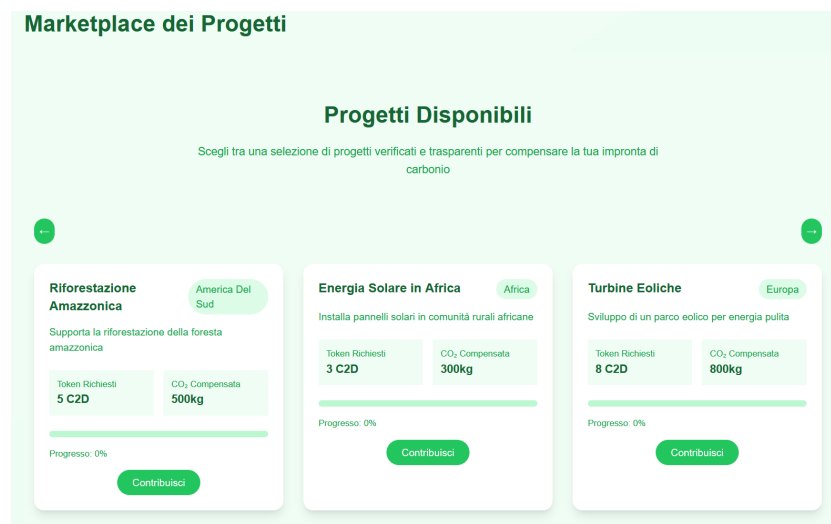


Figura 1.3: Sezione marketplace dei progetti

1.2 Lo standard ERC-20

Lo standard ERC-20 è uno standard tecnico utilizzato per i token fungibili sulla blockchain Ethereum. ERC sta per *Ethereum Request for Comments*, e il numero 20 è l'identificatore univoco di questa proposta. Questo standard definisce un insieme di eventi che un token deve implementare per essere considerato compatibile con ERC-20.

1.2.1 Funzioni Principali

Le funzioni principali definite dallo standard ERC-20 sono:

- `totalSupply()`: Restituisce il numero totale di token esistenti.
- `balanceOf(address account)`: Restituisce il saldo di token di un determinato indirizzo.
- `transfer(address recipient, uint256 amount)`: Trasferisce una quantità specificata di token all'indirizzo del destinatario.
- `allowance(address owner, address spender)`: Restituisce la quantità di token che il proprietario ha autorizzato a essere spesa dal `spender`.
- `approve(address spender, uint256 amount)`: Autorizza un indirizzo a spendere una quantità specificata di token per conto del proprietario.
- `transferFrom(address sender, address recipient, uint256 amount)`: Trasferisce token da un indirizzo a un altro, utilizzando il meccanismo di approvazione.

1.2.2 Eventi

Lo standard ERC-20 definisce anche due eventi principali:

- `Transfer(address from, address to, uint256 value)`: Emesso quando i token vengono trasferiti da un indirizzo a un altro.
- `Approval(address owner, address spender, uint256 value)`: Emesso quando un proprietario approva un `spender` per una determinata quantità di token.

Lo standard ERC-20 è fondamentale per l'interoperabilità dei token sulla blockchain Ethereum. Grazie a questo standard, i wallet e gli exchange possono supportare una vasta gamma di token in modo uniforme, semplificando la gestione e lo scambio di token.

2. Specifica dei Requisiti

L'applicazione Carbon2D che si intende realizzare dovrà soddisfare i seguenti requisiti.

1. **Connessione** al wallet Metamask.
2. **Deploy** dello Smart Contract sulla rete Sepolia.
3. **Calcolo** della propria impronta di carbonio attraverso apposito form.
4. **Acquisto** di token ERC-20:
 - deve permettere il mint dei token a seguito della richiesta di acquisto.
5. **Contribuzione** a progetti
 - i progetti devono essere visualizzabili all'interno di un marketplace
 - attraverso un pulsante sarà possibile trasferire i token dal proprio wallet al progetto a cui si vuole aderire
6. **Creazione** di progetti attraverso un pulsante dedicato, specificando:
 - titolo
 - descrizione
 - token richiesti
 - posizione geografica da menù a tendina
7. **Dashboard** da cui sarà possibile visualizzare
 - bilancio dei token posseduti
 - impronta di carbonio
 - CO_2 compensata

- numero di progetti a cui si è aderito
8. **Tabella** nella sezione dashboard in cui saranno presenti
- il tipo di interazione effettuata
 - la data in cui è stata effettuata
 - il numero di token
 - hash della transazione
9. **Link** alla piattaforma *etherscan.io* da cui sarà possibile visualizzare la transazione effettuata con tutti i dettagli.
10. **Interfaccia Utente:**
- design user-friendly
 - responsiva
 - compatibile con i browser più recenti

3. Progetto

Il progetto **Carbon2D** ha l'obiettivo di realizzare un sito web user-friendly e uno Smart Contract per l'implementazione del calcolo della propria impronta di carbonio, sfruttare i token acquistati per aderire a progetti già esistenti o crearne uno per la riduzione delle emissioni di CO_2 . Lo sviluppo della Dapp si è suddiviso in due fasi principali:

1. Sviluppo e deploy dello Smart Contract sulla blockchain locale Ganache
2. Sviluppo e deploy dello Smart Contract sulla blockchain Sepolia Ethereum Testnet

Abbiamo deciso di suddividerla in questo modo per una questione puramente sperimentale, a causa del difficile reperimento di grandi quantità di sepolia faucet. Il progetto presenta uno smart contract che implementa in maniera semplice e compatta i comportamenti descritti nella sezione 2, il contratto è stato scritto in linguaggio Turing complete **Solidity**. La parte di Frontend si elabora di diverse sezioni, per garantire un utilizzo semplice delle funzioni descritte nel contratto. L'interfaccia attraverso l'uso di pulsanti e descrizioni, rende facile l'utilizzo anche per chi non conosce affatto il mondo della blockchain. L'interazione tra sito e utente ha inizio connettendo il proprio wallet **Metamask**, abbiamo deciso di utilizzare un wallet per rendere l'interazione più realistica, senza passare dalla creazione di una moneta secondaria e alla possibilità di effettuare transazioni attraverso l'estensione fornita dai browser web come *Firefox*, *Chrome*,

La Web Dapp ha inoltre l'obiettivo di implementare una Dashboard riassuntiva di tutte le interazioni effettuate con essa, per **interazione** si intende:

- Acquisto di token
- Contribuzione a un progetto
- Creazione di un progetto

- Completamento e conseguente riscatto di un progetto creato

Inoltre forniamo all'utente un link che è rappresentato dall'hash della transazione effettuata, per visualizzare la transazione con il contratto dal sito web *etherscan.io* [5]. Viene fornita una descrizione dettagliata sia della transazione, ma anche del token, in funzione del numero di possessori e trasferimenti avvenuti.

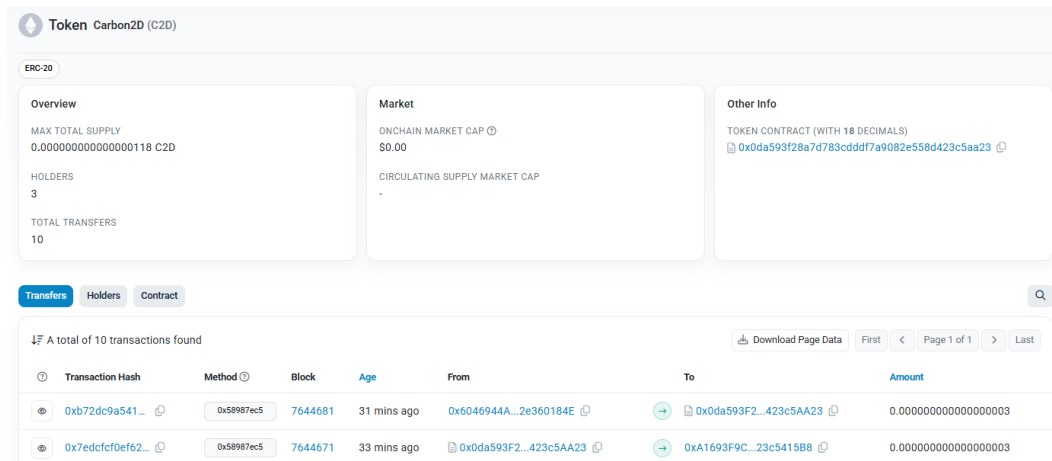


Figura 3.1: Pagina del Token "C2D" all'interno del sito web *etherscan.io*, con relativo esempio di transazioni

3.1 Frontend della WebDapp

Il frontend della DApp Carbon2D è costruito su Next.js, un framework React avanzato scelto per le sue capacità di ottimizzazione delle performance. Next.js offre diverse modalità di rendering che migliorano significativamente l'esperienza utente: il Server-Side Rendering genera le pagine dinamicamente sul server per una migliore indicizzazione e tempi di caricamento ridotti, mentre la Static Site Generation pre-genera le pagine statiche durante la build per massimizzare le performance. Queste tecnologie rendono l'applicazione veloce, reattiva e facilmente scalabile.

Per garantire un codice robusto e type-safe, l'applicazione è sviluppata in TSX, combinando TypeScript con JSX per una tipizzazione statica dei componenti React. Lo styling dell'interfaccia è gestito attraverso Tailwind CSS, un framework che permette di creare design responsivi e moderni con un approccio componibile. Queste tecnologie, unite alla gestione intelligente del rendering dinamico dei contenuti, rendono l'applicazione veloce, reattiva e facilmente scalabile.

3.1.1 Struttura del Progetto

Il progetto segue una struttura modulare che facilita la manutenzione e l'estensibilità del codice. Le principali cartelle sono:

- **app/**: Contiene le principali sezioni dell'applicazione, tra cui:
 - **Home Page**: Una panoramica iniziale dell'applicazione, in cui l'utente può esplorare le funzionalità disponibili e scegliere se calcolare l'impronta di carbonio, accedere al marketplace o consultare la propria dashboard.
 - **Calcolatore di Emissioni**: Un form che permette all'utente di inserire i dati necessari per stimare la propria impronta di carbonio. Dopo aver connesso il proprio wallet, l'utente può acquistare token per compensare l'impatto ambientale.
 - **Marketplace**: Un'area in cui gli utenti possono sia aggiungere nuovi progetti di compensazione delle emissioni, sia contribuire a progetti già esistenti.
 - **Dashboard**: Una sezione in cui l'utente può visualizzare:
 - * Il totale dei token posseduti
 - * I progetti a cui ha partecipato
 - * L'impronta di carbonio compensata
 - * Il totale dell'impronta calcolata
 - * Le transazioni effettuate per categoria, e la possibilità di aprire il dettaglio della transazione direttamente su testnet.
- **components/**: Racchiude componenti riutilizzabili come la barra di navigazione e sezioni interattive.
- **web3/**: Gestisce la comunicazione con la blockchain Ethereum tramite **Ether.js**. Contiene:
 - **config.ts**: File in cui sono definiti l'ABI (Application Binary Interface) e l'indirizzo dello smart contract sulla blockchain.
 - **contract.ts**: File che contiene le funzioni necessarie per interagire con lo smart contract, come:
 - * Connessione del wallet
 - * Acquisto di token
 - * Registrazione della compensazione di carbonio
 - * Recupero del saldo e delle transazioni

* Contributo ai progetti e creazione di nuovi progetti

- **lib/**: contiene la funzione `cn`, che combina le librerie `clsx` e `tailwind-merge` per gestire e ottimizzare dinamicamente le classi CSS, evitando conflitti tra le classi di Tailwind CSS.

3.2 Home Page e Navigazione

La homepage di Carbon2D accoglie gli utenti con un'interfaccia elegante e intuitiva. Al centro della pagina, una descrizione chiara introduce la missione della DApp: permettere agli utenti di calcolare e compensare la propria impronta di carbonio attraverso progetti sostenibili. La barra di navigazione, sempre presente nella parte superiore, offre un accesso immediato alle funzionalità principali: il calcolatore di emissioni per stimare il proprio impatto ambientale, il marketplace per esplorare e contribuire ai progetti di compensazione e la dashboard personale per monitorare le proprie attività. Un pulsante dedicato alla connessione del wallet completa l'interfaccia, permettendo agli utenti di iniziare immediatamente a interagire con la piattaforma blockchain.

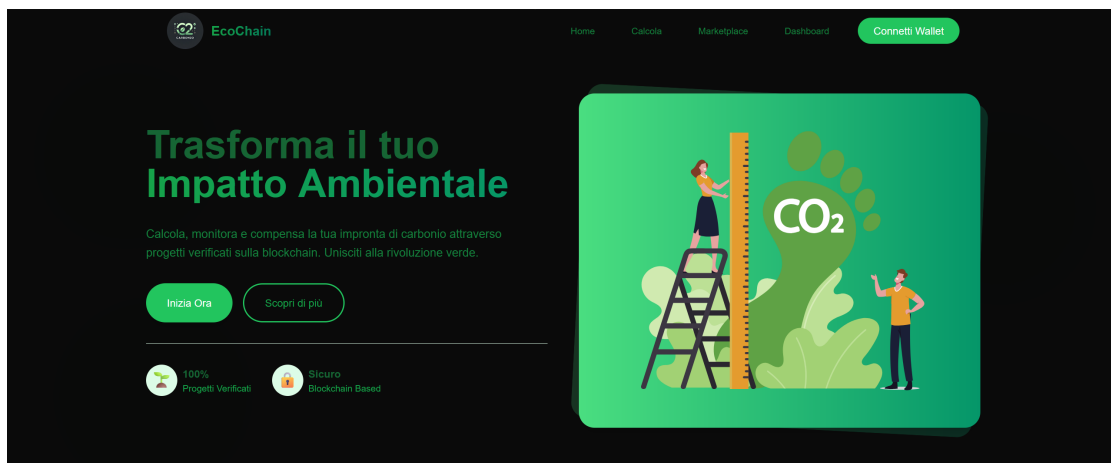


Figura 3.2: Home Page e Navigazione

3.3 Calcolatore delle Emissioni

Per calcolare la propria impronta di carbonio, l'utente deve compilare un form con dati specifici. Dopo aver connesso il proprio wallet, ha la possibilità di acquistare **C2D token** per compensare le emissioni.

The image shows a web interface for calculating a carbon footprint. The title is "Calcola la tua Impronta di Carbonio" in green. Below it, a subtitle reads "Scopri il tuo impatto ambientale e trasformalo in azione concreta attraverso i nostri green token". On the left, there is a form with four input fields: "Consumo elettrico" (2000 kWh), "Chilometri in auto" (15000 km), "Voli aerei" (2 voli), and "Consumo di carne" (90 kg). Each field has a small up/down arrow icon. At the bottom of the form is a green button labeled "Calcola Impronta". On the right, there are two summary boxes. The top one, titled "La tua impronta di carbonio annuale", shows "8700.00 kg CO₂" with a green progress bar. The bottom one, titled "Carbon2DToken Necessari", shows "87 C2D" and "Token necessari per la compensazione della tua impronta", with a blue button labeled "Acquista Token".

Calcola la tua Impronta di Carbonio

Scopri il tuo impatto ambientale e trasformalo in azione concreta attraverso i nostri green token

Consumo elettrico: 2000 kWh

Chilometri in auto: 15000 km

Voli aerei: 2 voli

Consumo di carne: 90 kg

Calcola Impronta

La tua impronta di carbonio annuale: **8700.00 kg CO₂**

Carbon2DToken Necessari: **87 C2D**

Token necessari per la compensazione della tua impronta

Acquista Token

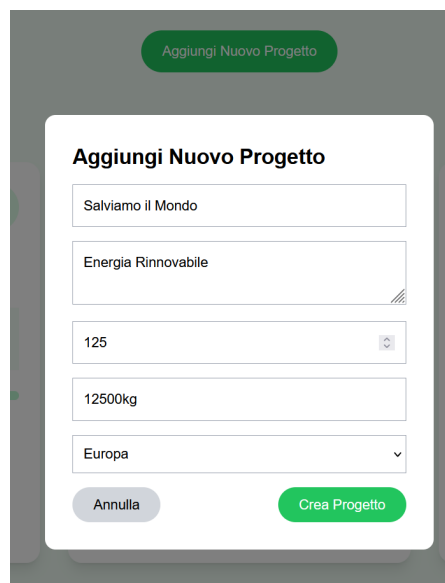
Figura 3.3: Calcolatore delle Emissioni

3.4 Marketplace

Come già specificato, il marketplace della piattaforma Carbon2D è progettato per facilitare sia la creazione di nuovi progetti di compensazione che la partecipazione a quelli esistenti.

Creazione di un nuovo progetto

Per creare un nuovo progetto, l'utente deve innanzitutto assicurarsi di aver collegato il proprio wallet alla piattaforma. Una volta effettuata la connessione, può cliccare sul pulsante **"Aggiungi Nuovo Progetto"** che aprirà un form dedicato. Nel form, l'utente inserisce il titolo del progetto, una descrizione dettagliata e il numero di token necessari. Un aspetto interessante è che la quantità di CO2 compensata viene calcolata automaticamente in base al numero di token inseriti, fornendo un feedback immediato sull'impatto ambientale del progetto. Infine, attraverso un menu a tendina, l'utente può specificare la posizione geografica del progetto. Una volta creato, il nuovo progetto diventa immediatamente visibile a tutti gli utenti della piattaforma.



The image shows a mobile application interface for creating a new project. At the top, there is a green button labeled "Aggiungi Nuovo Progetto". Below it, a white form titled "Aggiungi Nuovo Progetto" contains the following fields: a text input for the title "Salviamo il Mondo", a text input for the description "Energia Rinnovabile" with a small icon on the right, a numeric input for tokens "125" with a dropdown arrow, a text input for CO2 compensation "12500kg", and a dropdown menu for location "Europa". At the bottom of the form are two buttons: a grey "Annulla" button and a green "Crea Progetto" button.

Figura 3.4: Aggiunta di un nuovo Progetto

Contribuire a un progetto

La sezione dei progetti esistenti mostra tutti i progetti disponibili sulla piattaforma. Ogni progetto presenta una barra di avanzamento che indica in tempo reale la percentuale di completamento delle contribuzioni ricevute rispetto all'obiettivo stabilito. Per contribuire, l'utente deve essere connesso con il proprio wallet e possedere una quantità sufficiente di token C2D. Se non dispone di token, può acquistarli attraverso la sezione **"Calcola"** della piattaforma. Una volta in possesso dei token necessari, può selezionare il progetto desiderato e specificare quanto vuole contribuire. Il sistema verifica automaticamente che l'utente abbia un saldo sufficiente - ad esempio, se vuole contribuire con 5 token ma ne possiede solo 3, riceverà un messaggio che indica l'impossibilità di procedere. Anche in questo caso, tutti possono vedere la percentuale di completamento del progetto. Quando un progetto raggiunge il 100% delle contribuzioni necessarie, il pulsante **"Contribuisci"** viene sostituito da un'etichetta **"Completato"** non selezionabile.



The screenshot shows a web form titled "Contribuisci a Reforestazione Amazzonica". Below the title, there is a prompt: "Inserisci il numero di token con cui vuoi contribuire". A text input field contains the number "3". Below the input field, a green message states "Token rimanenti necessari: 5". At the bottom of the form, there are two buttons: a grey "Annulla" button and a green "Conferma Contributo" button.

Figura 3.5: Marketplace

3.5 Dashboard

La dashboard si presenta come un hub informativo completo delle attività dell'utente sulla piattaforma. Nella sezione superiore, un pannello di metriche chiave mostra immediatamente il saldo dei token C2D disponibili, l'impronta di CO2 totale calcolata attraverso il calcolatore, la quantità di CO2 già compensata tramite

i progetti supportati, e il numero totale di progetti a cui l'utente ha contribuito. La parte centrale della dashboard è dedicata alla cronologia delle transazioni, preceduta da una barra di filtri interattiva che permette di personalizzare la visualizzazione. Gli utenti possono scegliere di visualizzare tutte le transazioni o filtrare per categorie specifiche, come solo acquisti di token, solo contribuzioni ai progetti, o una combinazione di entrambi. Ogni transazione nella cronologia mostra informazioni essenziali come la tipologia dell'operazione (acquisto token, creazione progetto, contribuzione, completamento progetto), la data di esecuzione e i dettagli specifici dell'operazione. Ad esempio, per un acquisto di token vengono mostrati la quantità e il valore della transazione, mentre per una contribuzione si visualizza il nome del progetto e l'ammontare del contributo. Ogni transazione è accompagnata dal proprio hash, visualizzato come link cliccabile che reindirizza l'utente alla Sepolia TestNet per una verifica completa della transazione sulla blockchain.

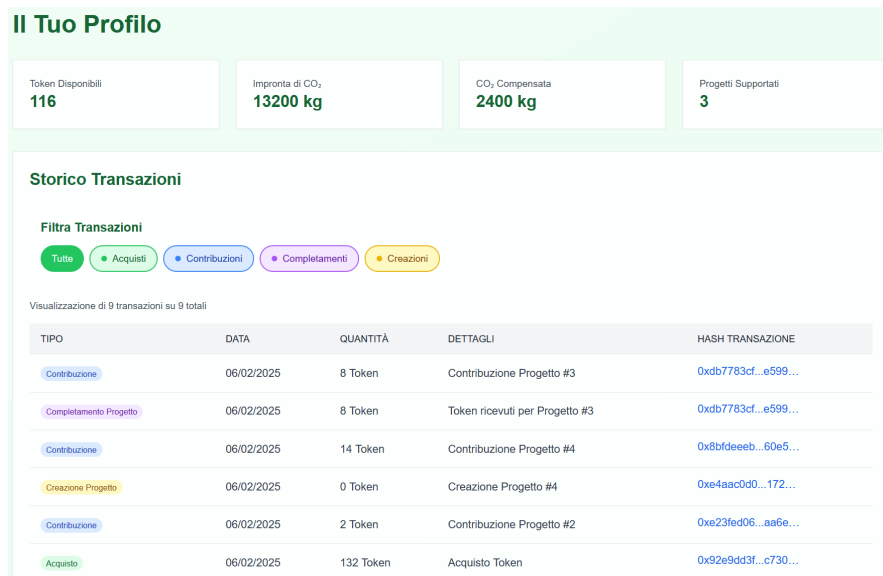


Figura 3.6: Dashboard

3.6 Integrazione Blockchain e Smart Contract

L'integrazione con la blockchain è implementata utilizzando ethers.js, una libreria moderna e completa che offre un buon supporto per la tipizzazione TypeScript. Attraverso MetaMask, gli utenti possono connettere i propri wallet in modo sicuro e interagire con la blockchain. Il cuore dell'interazione blockchain risiede nel file

contract.ts, che racchiude tutte le funzionalità necessarie per comunicare con lo smart contract.

- La funzione **getContract()** stabilisce la connessione iniziale, recuperando un'istanza del contratto associata al wallet dell'utente.
- Per gli acquisti di token, **buyCarbon2DTokens()** gestisce la transazione completa, inclusa la stima del gas e la gestione degli errori.
- Le funzioni **contributeToProject()** e **createNewProject()** facilitano rispettivamente la partecipazione e la creazione di progetti di compensazione.
- **recordUserEmissions()** registra sulla blockchain i dati delle emissioni calcolate, considerando vari parametri ambientali.
- Per monitorare lo stato dei progetti, **getProjectProgress()** fornisce aggiornamenti in tempo reale sul livello di completamento, mentre **calculateTokensNeeded()** determina precisamente quanti token sono necessari per una data compensazione di CO2.

Tutte queste interazioni sono rese possibili grazie alla configurazione in config.ts, dove l'ABI e l'indirizzo del contratto permettono una comunicazione diretta e sicura tra il frontend e la blockchain.

3.6.1 Smart Contract

Abbiamo scelto di implementare e fare il **deploy** del contratto utilizzando l'IDE web Remix [1]. L'IDE ci ha dato la possibilità di fare un deploy rapido sulla blockchain desiderata, sfruttando l'*injection* del wallet Metamask. Il contratto ha una struttura ordinata e si suddivide in due parti principali:

- Calcolo delle emissioni e loro relativa registrazione
- Contribuzione e creazione dei progetti

Il token **C2D** utilizza lo standard ERC-20, come riportato nella sezione 1.2 ci sono dei metodi obbligatori da implementare. L'IDE fornisce un ambiente di sviluppo intuitivo, attraverso un'interfaccia minimale; l'IDE implementa in maniera implicita tutti i metodi obbligatori che possono essere sovrascritti, in questo modo è possibile modificare i metodi per ottenere il comportamento desiderato. Il contratto è stato scritto in linguaggio Solidity, è necessario usare compilatori diversi a seconda della blockchain su cui si vuole fare il deploy:

- Ganache: compilatore 8.22
- Sepolia: compilatore 8.26+

Abbiamo notato questa differenza facendo alcuni test, quando effettuavamo il deploy su ganache con un compilatore superiore alla versione 8.22 riscontravamo problemi di funzionamento.

Nel contratto sono definite due strutture:

- `CompensationProject{}`
- `CarbonData{}`

nel seguito verrà fornita una descrizione approfondita di entrambe.

CompensationProject{}

La struttura rappresenta i progetti di *compensazione ambientale*, gli attributi di tale struttura servono per definire le caratteristiche che un progetto dovrebbe avere 3.7.

Grazie all'utilizzo della variabile `projectCounter`, si ha un contatore che tiene memoria del numero di progetti creati, tale variabile è utile in fase di visualizzazione e di gestione degli indici per un controllo coerente sia della contribuzione sia in fase di completamento degli stessi. Per riuscire a implementare la creazione dei progetti, abbiamo deciso di creare due funzioni con il nome:

```

contract Carbon2D is ERC20, Ownable {
    // Struttura che definisce un progetto di compensazione ambientale
    struct CompensationProject {
        string name;           // Nome del progetto
        string description;     // Descrizione del progetto
        string location;        // Posizione del progetto
        uint256 requiredTokens; // Numero di token necessari per completare il progetto
        uint256 co2Reduction;   // Quantità di CO2 che il progetto ridurrà (in kg)
        bool active;            // Stato del progetto (attivo/completato)
        uint256 totalContributed; // Totale dei token già contribuiti
        address projectOwner;   // Indirizzo del creatore del progetto
    }
}

```

Figura 3.7: Struttura che definisce i progetti

- `_createProject()`
- `createProject()`

La differenza sostanziale tra le due è la visibilità, la prima che abbiamo definito ha lo scopo di aggiornare il contatore e inizializzare i parametri definiti nella struttura, la sua visibilità è solo **interna** al progetto. La seconda funzione invece è **pubblica**, infatti può essere chiamata dall'esterno, si devono passare tre parametri:

- nome
- descrizione del progetto
- posizione geografica del progetto
- token richiesti
- CO_2 compensata

La funzione chiama a sua volta la funzione interna per espletare il suo compito ed emettere l'evento `ProjectCreated()` che può essere ascoltato dal client e utilizzato per la visualizzazione sull'interfaccia web, al termine della creazione di un progetto. Abbiamo poi definito la funzione che rappresenta l'adesione e la relativa compen-

```

// Funzione per creare un nuovo progetto (pubblica)
function createProject(
    string memory name,
    string memory description,
    string memory location,
    uint256 requiredTokens,
    uint256 co2Reduction
) public {
    _createProject(name, description, location, requiredTokens, co2Reduction);
}

```

Figura 3.8: Funzione `createProject()`

sazione dei progetti esistenti, che abbiamo chiamato `compensateProject()`, è un tipo di funzione **pubblica**.

compensateProject()

Prende come parametri, l'ID del progetto, e il numero di token con cui si vuole contribuire. Sono presenti tre controlli principali:

- se l'ID del progetto è presente
- se il progetto è attivo attraverso l'attributo `active`
- se i token posseduti dal contribuente sono sufficienti usando la funzione `balanceOf()`

```
function compensateProject(uint256 projectId, uint256 tokenAmount) public {  
    require(projectId > 0 && projectId <= projectCounter, "Invalid project ID");  
    require(projects[projectId].active, "Project not active");  
    require(balanceOf(msg.sender) >= tokenAmount, "Insufficient tokens");  
  
    CompensationProject storage project = projects[projectId];
```

Figura 3.9: Funzione `compensateProject()`

Inoltre questa funzione gestisce anche la situazione in cui il numero totale di token richiesti per il progetto viene raggiunto: come deciso in fase di definizione dei requisiti, al raggiungimento del numero dei token richiesti si inviano i token collezionati nel contratto al proprietario del progetto, e viene settato l'attributo del progetto `active = false`.

```
// Se il progetto raggiunge il target, viene completato  
if (project.totalContributed >= project.requiredTokens) {  
    project.active = false;  
    _transfer(address(this), project.projectOwner, project.totalContributed);  
    emit ProjectCompleted(projectId, project.projectOwner, project.totalContributed);  
}
```

Figura 3.10: Check del completamento e conseguente invio dei token al proprietario del progetto

CarbonData{}

La struttura 3.11 rappresenta i dati dell'impronta di carbonio dell'utente, al cui interno gli attributi sono i corrispettivi parametri usati per il calcolo della *carbon footprint*.

Questa struttura viene usata all'interno della funzione `recordEmissions()`, tale funzione viene usata per registrare il valore corrispondente in kg di CO_2 delle emissioni dell'utente. Al suo interno viene usata la funzione `calculateEmissions()`

```
// Struttura per memorizzare i dati dell'impronta di carbonio di un utente
struct CarbonData {
    uint256 electricityConsumption; // Consumo di elettricità
    uint256 carKilometers;          // Chilometri percorsi in auto
    uint256 flights;                 // Numero di voli
    uint256 meatConsumption;         // Consumo di carne
    uint256 totalEmissions;          // Totale delle emissioni calcolate
    uint256 timestamp;               // Timestamp dell'ultima registrazione
    uint256 tokenPrice;              // Prezzo del token al momento della registrazione
}
```

Figura 3.11: Struttura `CarbonData{}`

che ha uno scopo puramente(**pure**) computazionale applicando dei fattori di conversione per il calcolo della *footprint*.

Dopo aver calcolato la propria impronta è possibile acquistare i token corrispondenti attraverso l'utilizzo della funzione `buyTokens()`.

buyTokens()

Una funzione pubblica che prende come parametro la quantità di token desiderata. Abbiamo deciso di fare un `_mint` dei token ogni volta che un utente li vuole acquistare, l'acquisto è possibile grazie all'utilizzo della criptovaluta ETH. Al termine del riscatto dei token viene emesso l'evento `TokensMinted()`.

```
function buyTokens(uint256 tokenAmount) public payable {
    require(msg.value >= TOKEN_PRICE * tokenAmount, "Insufficient payment");
    _mint(msg.sender, tokenAmount);
    totalMintedTokens += tokenAmount;
    emit TokensMinted(msg.sender, tokenAmount, 0);
}
```

Figura 3.12: Funzione `buyTokens()`

I token C2D seguono lo standard ERC-20, come descritto in precedenza, il costruttore ha la caratteristica di inizializzare le variabili che fungono da contatori. Vengono inizializzati anche i progetti che la nostra Dapp presenta di default. Abbiamo deciso di inserire tre progetti standard, sia per una questione di test, sia per dare la possibilità a un nuovo utente di visualizzarli nella pagina marketplace. L'utente in questo modo può aderire a un progetto e non è costretto a doverne per forza creare uno. I progetti di default appartengono a chi ha fatto il deploy del contratto. Inoltre viene utilizzato il contratto `Ownable.sol` per la gestione dei permessi, e per definire il concetto di **proprietario**; viene utilizzato per esempio per poter convertire i token posseduti in ETH, attraverso la funzione `withdraw()`.

```

constructor() ERC20("Carbon2D", "C2D") Ownable(msg.sender) {
    projectCounter = 0;
    totalMintedTokens = 0;
    // Crea i progetti iniziali con valori predefiniti
    _createProject("Riforestazione Amazzonica", 5, 500);
    _createProject("Energia Solare in Africa", 3, 300);
    _createProject("Turbine Eoliche", 8, 800);
}

```

Figura 3.13: Costruttore del contratto Carbon2D.sol

3.7 Problemi Riscontrati

Si riportano di seguito i principali problemi riscontrati nello svolgimento del progetto Carbon2D.

- Creazione dei progetti
- Contribuzione ai progetti
- Stato di completamento di progetto attraverso una barra di progresso

Questi sono alcuni dei problemi che abbiamo riscontrato durante lo sviluppo del nostro progetto. Come descritto nella sezione 1, l'app ha l'obiettivo di far spendere i token acquistati in progetti per la riduzione delle emissioni di CO_2 . I progetti standard definiti nella sezione marketplace sono 3 come descritto nella sezione 3.6.1, per poter aggiungere un nuovo progetto abbiamo dato la possibilità all'utente di poterlo creare attraverso un pulsante dedicato. Tale creazione deve essere coerente ai progetti già presenti, i progetti non dovevano essere creati e memorizzati solo nella pagina web, ma anche a livello di smart contract, perchè devono essere riconosciuti come **transazione**. Abbiamo risolto tale problema attraverso la funzione `_createProject()` e un evento associato, in cui si memorizza un contatore relativo al numero di progetti presenti e al numero di token richiesti.

Il problema di contribuzione è stato riscontrato, quando dovevamo decidere in che modo i token spesi venissero accreditati; se inviare i token direttamente al proprietario del progetto anche prima del completamento dello stesso o utilizzare un'entità come intermediario. Abbiamo scelto di risolvere questo dilemma sfruttando il **balance** del contratto, cioè accreditando i token direttamente sul contratto fino al completamento del progetto, cioè fino al raggiungimento dei token richiesti. In questo modo, al completamento del progetto, i token accumulati vengono inviati direttamente al proprietario/creatore del progetto.

Un altro problema incontrato riguarda la **progress bar**. Abbiamo voluto fornire una visualizzazione grafica dello stato di completamento di un progetto, è stato complicato in termini di codice *javascript*. Attraverso l'interazione con la blockchain con l'utilizzo della libreria *ethers.js*, dovevamo gestire il numero di token che il progetto richiedeva e il numero di token ricevuti fino a quel momento. Abbiamo sistemato tale problema implementando una funzione all'interno dello smart contract, che abbiamo chiamato *getProjectProgress*. La funzione viene chiamata all'interno del file `contract.ts`, fornendo come unico parametro l'**ID** del progetto. Tale funzione recupera il progetto salvato in memoria e verifica due condizioni:

- se il progetto ha un numero di token richiesti pari a 0
- se il contributo ricevuto è maggiore o uguale ai token richiesti

In questo modo abbiamo gestito le due casistiche limite; invece, negli altri casi veniva applicata la formula:

$$(\text{project.totalContributed} * 100) / \text{project.requiredTokens}$$

4. Conclusioni e sviluppi futuri

La WebDapp sviluppata offre un innovativo portale che consente agli utenti di calcolare la propria impronta di carbonio, riscattare token in funzione di essa e utilizzarli per aderire a progetti presenti nella sezione marketplace. Il progetto permette anche la creazione di iniziative personali, interagendo in modo rapido e responsivo con la blockchain tramite il wallet Metamask, sfruttando la libreria `ethers.js`. Le interazioni effettuate con l'applicazione sono tracciabili attraverso la sezione dashboard, nonché sul sito etherscan.io [5], dove è possibile visualizzare i dettagli delle transazioni.

Tra gli sviluppi futuri¹, prevediamo di implementare uno script che permetta l'acquisto indipendente dei token, senza necessità di calcolare preventivamente la propria impronta. L'obiettivo è estendere l'uso del nostro token su altre piattaforme, per trasformarlo in uno standard globale nella rappresentazione della CO₂ e contribuire alla sua riduzione. Inoltre, ci proponiamo di abilitare anche altre criptovalute come opzioni per l'acquisto del token C2D, ampliando così le possibilità di utilizzo. Seguendo l'esempio di importanti associazioni, non fermarsi solo all'obiettivo di ridurre la CO₂, ma anche creare nuovi token per il finanziamento di progetti per la rigenerazione di ambienti marini e per l'accrescimento di fonti energetiche pulite, in termini di utilizzo [6]. Rendere il sito disponibile pubblicamente, così da poter raggiungere più persone possibili, sensibilizzando sul tema delle emissioni e anche di contribuzione a progetti che possono aiutare a ridurre in maniera significativa la CO₂ presente sul nostro pianeta.

¹Codice sorgente disponibile su GitHub: <https://github.com/Daniele-00/carbon-dapp.git>

5. Bibliografia

- [1] Ethereum Foundation. Remix - ethereum ide. <https://remix.ethereum.org/>, 2025. Consultato il: 5-febbraio-2025.
- [2] Richard Moore. ethers.js: A complete ethereum wallet implementation and utilities in javascript (and typescript). <https://docs.ethers.io/>, 2023. Versione 6.x, Consultato il: 5-febbraio-2025.
- [3] PixelPlex. Carbon credit tokenization: Unlocking the future of sustainable finance. <https://pixelplex.io/blog/carbon-credit-tokenization/>, 2025. Consultato il: 5-febbraio-2025.
- [4] Green Future Project. Green future project. <https://greenfutureproject.com/>, 2025. Consultato il: 5-febbraio-2025.
- [5] sepolia.etherscan.io. Testnet Sepolia (ETH) Blockchain Explorer, 2025. [Online <https://sepolia.etherscan.io/>; Consultato il: 5-febbraio-2025].
- [6] Maritime Carbon Solutions. Tokenization of carbon emissions. <https://www.maritimecarbonsolutions.com/insights/tokenization-of-carbon-emissions>, 2025. Consultato il: 5-febbraio-2025.
- [7] F. Vogelsteller and V. Buterin. ERC-20: Token Standard | eips.ethereum.org, 2015. [Online <https://eips.ethereum.org/EIPS/eip-20>; Consultato il: 5-febbraio-2025].
- [8] F. Vogelsteller and V. Buterin. ERC-20 Token Standard | ethereum.org, 2015. [Online <https://ethereum.org/en/developers/docs/standards/tokens/erc-20>; Consultato il: 5-febbraio-2025].